

Mechi Multiple Campus
(Tribhuvan University)
Bhadrapur, Jhapa.



Lab Report of Operating System(CACS-251)
Faculty of Humanities and Social Sciences
Tribhuvan University
Kritipur, Nepal.

Submitted by:

Pawan Adhikari

Roll no: - 31

Submitted to:

Mechi Multiple Campus

Department of Bachelor in Computer Application
Bhadrapur, Jhapa, Nepal.

Internal Examiner

External Examiner

LAB SHEET- 1: PROCESS SCHEDULING

1. C- Program to Implement First Come First Serve Scheduling Algorithm(FCFS):

```
#include<stdio.h>

int main()
{
    int n,bt[20],wt[20],tat[20],av_wt=0,av_tat=0,i,j;
    printf("Enter total number of processes(maximum 20):");
    scanf("%d",&n);
    printf("\nEnter Process Burst Time\n");
    for(i=0;i<n;i++)
    {
        printf("P[%d]:",i+1);
        scanf("%d",&bt[i]);
    }
    wt[0]=0;
    for(i=1;i<n;i++)
    {
        wt[i]=0;
        for(j=0;j<i;j++)
            wt[i]+=bt[j];
    }
    printf("\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time");
    for(i=0;i<n;i++)
    {
        tat[i]=bt[i]+wt[i];
        av_wt+=wt[i];
        av_tat+=tat[i];
        printf("\nP[%d]\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt[i],tat[i]);
    }
    av_wt/=i;
```

```

    av_tat/=i;

    printf("\n\nAverage Waiting Time:%d",av_wt);

    printf("\n\nAverage Turnaround Time:%d",av_tat);

    return 0;

}

```

Output:

```

Enter total number of processes(maximum 20):4

Enter Process Burst Time
P[1]:5
P[2]:3
P[3]:8
P[4]:6

Process      Burst Time    Waiting Time    Turnaround Time
P[1]          5             0              5
P[2]          3             5              8
P[3]          8             8             16
P[4]          6            16             22

Average Waiting Time:7
Average Turnaround Time:12
-----

```

2. C- Program to Implement Shortest Job First Scheduling Algorithm(SJF):

```
#include<stdio.h>

void main()
{
    int bt[20],p[20],wt[20],tat[20],i,j,n,total=0,pos,temp;
    float avg_wt,avg_tat;
    printf("Enter number of process:");
    scanf("%d",&n);
    printf("\nEnter Burst Time:\n");
    for(i=0;i<n;i++)
    {
        printf("p%d:",i+1);
        scanf("%d",&bt[i]);
        p[i]=i+1;        //contains process number
    }
    for(i=0;i<n;i++)
    {
        pos=i;
        for(j=i+1;j<n;j++)
        {
            if(bt[j]<bt[pos])
                pos=j;
        }
        temp=bt[i];
        bt[i]=bt[pos];
        bt[pos]=temp;
        temp=p[i];
        p[i]=p[pos];
        p[pos]=temp;
    }
}
```

```

wt[0]=0;
for(i=1;i<n;i++)
{
    wt[i]=0;
    for(j=0;j<i;j++)
        wt[i]+=bt[j];
    total+=wt[i];
}
avg_wt=(float)total/n;
total=0;
printf("\nProcess\t Burst Time \tWaiting Time\tTurnaround Time");
for(i=0;i<n;i++)
{
    tat[i]=bt[i]+wt[i];
    total+=tat[i];
    printf("\np%d\t\t %d\t\t %d\t\t%d",p[i],bt[i],wt[i],tat[i]);
}
avg_tat=(float)total/n;
printf("\n\nAverage Waiting Time=%f",avg_wt);
printf("\n\nAverage Turnaround Time=%f\n",avg_tat);
}

```

Output:

```

Enter number of process:4
Enter Burst Time:
p1:5
p2:3
p3:8
p4:6
Process      Burst Time      Waiting Time      Turnaround Time
p2           3              0                 3
p1           5              3                 8
p4           6              8                14
p3           8             14                22
Average Waiting Time=6.250000
Average Turnaround Time=11.750000
-----

```

3. C-program to Implement Priority Scheduling Algorithm:

```
#include<stdio.h>

int main()
{
    int bt[20],p[20],wt[20],tat[20],pr[20],i,j,n,total=0,pos,temp,avg_wt,avg_tat;

    printf("Enter Total Number of Process:");

    scanf("%d",&n);

    printf("\nEnter Burst Time and Priority\n");

    for(i=0;i<n;i++)
    {
        printf("\nP[%d]\n",i+1);

        printf("Burst Time:");

        scanf("%d",&bt[i]);

        printf("Priority:");

        scanf("%d",&pr[i]);

        p[i]=i+1;
    }

    for(i=0;i<n;i++)
    {
        pos=i;

        for(j=i+1;j<n;j++)
        {
            if(pr[j]<pr[pos])

                pos=j;
        }

        temp=pr[i];

        pr[i]=pr[pos];

        pr[pos]=temp;
    }
}
```

```

    temp=bt[i];
    bt[i]=bt[pos];
    bt[pos]=temp;

    temp=p[i];
    p[i]=p[pos];
    p[pos]=temp;
}
wt[0]=0;
for(i=1;i<n;i++)
{
    wt[i]=0;
    for(j=0;j<i;j++)
        wt[i]+=bt[j];
    total+=wt[i];
}
avg_wt=total/n;
total=0;
printf("\nProcess\t Burst Time \tWaiting Time\tTurnaround Time");
for(i=0;i<n;i++)
{
    tat[i]=bt[i]+wt[i];
total+=tat[i];

    printf("\nP[%d]\t\t %d\t\t %d\t\t\t%d",p[i],bt[i],wt[i],tat[i]);
}
avg_tat=total/n;
printf("\n\nAverage Waiting Time=%d",avg_wt);
printf("\n\nAverage Turnaround Time=%d\n",avg_tat);
return 0;
}

```


Output:

Enter Total Number of Process:5

Enter Burst Time and Priority

P[1]

Burst Time:10

Priority:3

P[2]

Burst Time:1

Priority:1

P[3]

Burst Time:2

Priority:3

P[4]

Burst Time:1

Priority:4

P[5]

Burst Time:5

Priority:2

Process	Burst Time	Waiting Time	Turnaround Time
P[2]	1	0	1
P[5]	5	1	6
P[3]	2	6	8
P[1]	10	8	18
P[4]	1	18	19

Average Waiting Time=6

Average Turnaround Time=10

4. C-program to Implement Round Robin Scheduling Algorithm:

```
#include<stdio.h>

int main()
{
    int count,j,n,time,remain,flag=0,time_quantum;
    int wait_time=0,turnaround_time=0,at[10],bt[10],rt[10];
    printf("Enter Total Process:\t ");
    scanf("%d",&n);
    remain=n;
    for(count=0;count<n;count++)
    {
        printf("Enter Arrival Time and Burst Time for Process Process Number %d :",count+1);
        scanf("%d",&at[count]);
        scanf("%d",&bt[count]);
        rt[count]=bt[count];
    }
    printf("Enter Time Quantum:\t");
    scanf("%d",&time_quantum);
    printf("\n\nProcess\t|Turnaround Time|Waiting Time\n\n");
    for(time=0,count=0;remain!=0;)
    {
        if(rt[count]<=time_quantum && rt[count]>0)
        {
            time+=rt[count];
            rt[count]=0;
            flag=1;
        }
        else if(rt[count]>0)
        {
            rt[count]-=time_quantum;
```

Output:
